# Automated Generation of Intrusion Detection Rules Using Generative AI in a Rule-Based IDS Deployment

A report submitted in fulfillment of the requirements for the award of the degree of Bachelor of Information Technology

**Group 13**


| Student | ID |
|---|---|
| Bigyan Siwakoti | S386010 |
| Irab Baral | S387115 |
| Binay Siwakoti | S387596 |
| Shahswat Acharya | S391214 |


**Supervisor: Prof. Bharanidharan Shanmugam**

Due date: 18 October 2026

Declaration

I hereby declare that the work herein, now submitted as a final report for the degree of Bachelor of Information Technology at Charles Darwin University, is the result of my own investigations, and all references to ideas and work of other researchers have been specifically acknowledged. I hereby certify that the work embodied in this report has not already been accepted in substance for any degree and is not being currently submitted in candidature for any other degree.

Signature: __binay_bigyan_irab_shahswat_____

Date: 18 October 2026

# Contents

Contents....4
List of Figures....6
List of Tables....7
List of Abbreviations....8
Abstract....9
I. Introduction....10
Background....10
Project Scope....10
Aim of Research....10
Structure of Report....11
II. Literature Review....12
Introduction of literature review....12
Intrusion detection and the rule-authoring bottleneck....12
Generative AI and agents for detection rules....13
Evaluation datasets and label quality....14
Detection-engineering practice....14
Conclusions from the literature review....14
III. Approach....15
System architecture....15
Agentic loop and provider adapter....16
The agent's tools....16
Technology stack....17
Safety design and evaluation integrity....17
Replicating detection-engineering practice: the hunt condition....19
Experiments....20
IV. Execution....21
Datasets....21
CIC-IDS2017 triage....21
UNSW-NB15 triage....22
Strengthening the human baseline....22
AI rounds on the strong baseline (Experiment 3)....23
V. Analysis and Discussion of Results....25
Rule quality against stock rules (Experiment 2)....25
What context the model needs (Experiment 1)....26
Against a strong baseline: attribution and the topology trap....27
Hold-out validation....28
Discussion....29

VI. Conclusions.....31
VII. Lessons Learned.....32
VIII. Future Work.....33
References.....34
Appendix A: Turnitin Similarity Report.....38
Appendix B: AI Declaration Form.....39
Appendix C: Data and Source Code.....40
Appendix D: Supplementary Material.....41

# List of Figures

Figure 2-1: Taxonomy of intrusion detection approaches....12
Figure 3-1: System architecture....15
Figure 3-2: The agentic tool-calling loop....16
Figure 3-3: The agent's toolkit and engine-enforced safeguards....17
Figure 3-4: Safety asymmetry between detection and suppression....18
Figure 3-5: Experiment profiles....18
Figure 3-6: The hunt condition procedure....19
Figure 3-7: The test_rule anti-topology gate....20
Figure 4-1: Alert volume per capture (log scale)....21
Figure 4-2: Categories identified per UNSW-NB15 run....22
Figure 4-3: Strengthening the UNSW-NB15 baseline....23
Figure 5-1: CIC-IDS2017 flow-level detection by attack....25
Figure 5-2: Precision of stock and AI-authored rules....26
Figure 5-3: Attack categories covered....26
Figure 5-4: Experiment 1: full context versus alerts only....27
Figure 5-5: AI-round gains: apparent, attributable and genuine....28
Figure 5-6: AI-authored rule classes by condition....28
Figure 5-7: Genuine gain on development and hold-out captures....29
Figure 5-8: Detection against three denominators....30
Figure D-1: Status of every AI-authored rule definition....42
Figure D-2: Alerts of each hunt rule on each development capture, grouped by the capture it was written on....42
Figure D-3: Selected rule families versus enabling every disabled rule — detection and alerts on non-attacker hosts....43
Figure D-4: Attack-like activity the hunt reports left without a rule, grouped by cause....43

## List of Tables

Table 2-1: LLM rule-generation systems compared....13
Table 3-1: Technology stack....17
Table 3-2: The three experiments....20
Table 4-1: Datasets used....21
Table 4-2: AI rounds on baseline v2 (14,935 events)....23
Table 5-1: Experiment 1 results....27
Table 5-2: Frozen AI rules on the hold-out (4,634 events)....29
Table C-1: Repository layout....40
Table D-1: The agent's tools....41
Table D-2: Audit of AI-authored rule definitions (Experiments 1–2)....41
Table D-3: CIC-IDS2017 execution summary....41

# List of Abbreviations

| Abbreviation | Meaning |
|---|---|
| ADS | Alerting and Detection Strategy (Palantir framework) |
| API | Application programming interface |
| C2 | Command and control |
| CVE / NVD | Common Vulnerabilities and Exposures / National Vulnerability Database |
| DoS / DDoS | (Distributed) denial of service |
| ET Open | Emerging Threats Open community ruleset |
| eve.json | Suricata's newline-delimited JSON event log |
| GT | Ground truth (dataset labels) |
| HOME_NET | The protected address ranges in the Suricata configuration |
| IDS / NIDS | (Network) intrusion detection system |
| LLM | Large language model |
| ML | Machine learning |
| pcap | Packet capture file |
| SID | Suricata/Snort rule identifier |
| SOC | Security operations centre |
| SSE | Server-Sent Events |
| UI | User interface |

# Abstract

A signature-based intrusion detection system (IDS) detects only what its rules describe, and every rule is written by a human analyst who investigates alerts, isolates what is unique to an attack, and writes and tests the rule. This project asks whether a large language model (LLM) can take over that analyst role, and what context it needs. We built an agentic triage engine that gives a hosted LLM controlled tools over Suricata's output and lets it write and replay-validate rules, while detection itself stays rule-based. Scored against ground truth on CIC-IDS2017 and UNSW-NB15, the model identified 13 of 14 and 8 of 9 attack categories, and its rules raised UNSW-NB15 detection over stock ET Open from 9.2% to 15.5%. Against a baseline strengthened as a detection engineer would, triage-driven rules added under half a percentage point; a workflow replicating detection-engineering practice added 1.2 points, and 1.5 on untouched hold-out traffic with no false alarms. The answer is split: yes for triage, partly for detection.



*Keywords:* intrusion detection, generative AI, Suricata, rule generation, agentic triage, detection engineering

# I. Introduction

## Background

Intrusion detection systems monitor network traffic for signs of attack. Signature-based systems such as Snort and Suricata remain the most widely deployed open-source approach because every alert traces back to a human-readable rule (Roesch 1999; Khraisat et al. 2019). Their weakness has been known since the earliest work on security monitoring: if no rule describes an attack, the attack is not detected (Anderson 1980).

Writing a rule is skilled, manual work. An analyst investigates the traffic, identifies what distinguishes the attack, writes the rule in the engine's syntax, tests it against attack and benign traffic and deploys it — a cycle that takes hours to days of scarce expertise (Scarfone & Mell 2007). Large language models can now read structured data, call tools and write formal text, which raises the question of whether this analyst loop can be automated without giving up deterministic, inspectable detection.

## Project Scope

The project is bounded by the supervisor's constraints. Only open-source or open-access software is used. Machine learning is not used for detection: the LLM is an automation layer that writes rules for the Suricata engine and never classifies traffic itself. The IDS is network-based and works offline, replaying public packet captures (CIC-IDS2017 and UNSW-NB15) rather than monitoring a live network. Zero-day detection and host-based sensors are out of scope.

## Aim of Research

**Research question:** *can a large language model take over the analyst's role in the alert-handling loop of a rule-based IDS, and what context does it need to do that job well?* The project has four objectives:

- **O1 — Build** an agentic triage engine that lets an LLM investigate Suricata alerts, write detection rules and validate them by replay, with human approval for any change that reduces detection.
- **O2 — Measure** every AI-authored rule against ground-truth labels on two datasets, one never used in development.
- **O3 — Test context** by comparing triage with alert events only against triage with full sensor context.
- **O4 — Test practice** by replicating real detection-engineering practice and measuring what it adds over the strongest human-tuned baseline we could build, on held-out traffic.

## Structure of Report

Section II reviews the literature. Section III describes the design: architecture, tools, technology stack and experiments. Section IV reports the execution on both datasets, and Section V analyses the results against the objectives. Sections VI to VIII give the conclusions, lessons learned and future work.

# II. Literature Review

## Introduction of literature review

The review is organised thematically: intrusion detection foundations and the rule-authoring bottleneck; generative AI and agents for security rules; evaluation datasets; and detection-engineering practice. Each theme fed a design decision in Section III.

## Intrusion detection and the rule-authoring bottleneck

Denning (1987) formalised intrusion detection and the split between misuse detection, which matches known attack patterns, and anomaly detection, which flags deviations from normal behaviour; specification-based detection later added explicit models of allowed behaviour (Ko, Ruschitzka & Levitt 1997; Sekar et al. 2002). Figure 2-1 summarises the taxonomy. Signature systems such as Snort, Suricata and Bro/Zeek are deterministic and auditable (Roesch 1999; Paxson 1999; Scarfone & Mell 2007). Anomaly and machine-learning detectors can find novel attacks but produce high false-positive rates and decisions that are hard to explain (Garcia-Teodoro et al. 2009; Liao et al. 2013; Aljanabi, Ismail & Ali 2021); the base-rate fallacy makes even accurate detectors noisy in practice (Axelsson 2000).

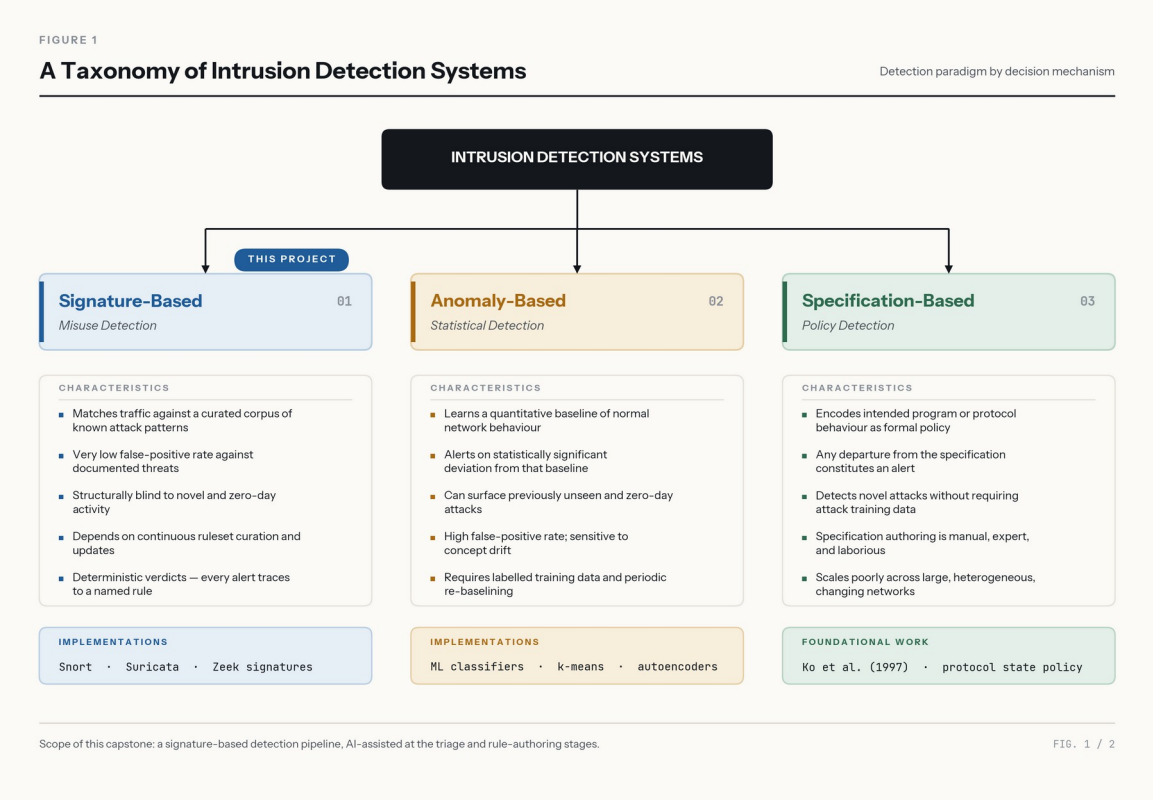



Figure 2-1: Taxonomy of intrusion detection approaches

Sommer and Paxson (2010) explain why machine learning has struggled in operational intrusion detection — scarce representative training data, costly errors and a semantic gap between statistical anomalies and security meaning — and recommend automation that supports analysts. Surveys agree that the limiting factor for signature IDS is the human effort and domain knowledge needed to write rules (Buczak & Guven 2016; Khraisat et al. 2019; Ferrag et al. 2020). Community rulesets such as ET Open, with over 52,000 rules, reduce that effort but are reactive and leave deployment-specific gaps (Proofpoint 2026).

The bottleneck is one of knowledge acquisition rather than technology. An analyst knows implicitly what separates attack traffic from legitimate traffic and must make that knowledge explicit in the engine's syntax, then prove it on both kinds of traffic. Specification-based systems show the same cost in another form: their models of allowed behaviour must be maintained by hand whenever the protected system changes (Sekar et al. 2002). Any automation that reduces this cost without giving up inspectable rules addresses the main practical weakness of signature detection, which is the opening this project explores.

## Generative AI and agents for detection rules

Transformer language models (Vaswani et al. 2017) can produce formally structured output, and agent designs let them interleave reasoning with tool calls and learn when to use tools (Yao et al. 2023; Schick et al. 2023; Anthropic 2025). Their main risk is hallucination: fluent, confident output not supported by the evidence (Ji et al. 2023). In an agent that can edit rules, a hallucination becomes a wrong action. Table 2-1 compares recent LLM rule-generation systems with this project.

Table 2-1: LLM rule-generation systems compared

| System | Starts from | Validation | Network IDS rules |
|---|---|---|---|
| Zhang et al. (2025) | Malicious software packages | Detection of known samples | No (package rules) |
| FALCON (Mitra et al. 2025) | Threat-intelligence reports | Syntax, semantic, performance checks | Snort + YARA |
| GRIDAI (Li et al. 2025) | Labelled attack samples | Multi-agent repair | Yes |
| GenTI (Hadi, Yasmin & Shoker 2026) | Curated payloads | Benchmark | Yes |
| RuleAutoPilot (Rehman, Piplai & Kantarcioglu 2026) | Malware traffic | Loads, fires, silent on benign | Suricata |
| This project | Unlabelled sensor output | Replay; anti-topology gate; GT scoring | Suricata |

Two findings shaped our design. RuleAutoPilot shows that rules must be validated by executing them — its execution-grounded checks raised F1

from 0.443 to 0.539 (Rehman, Piplai & Kantarcioglu 2026). A study of security experts found that they treat LLM-written network rules as drafting aids, citing logic errors in 12% of cases (di Filippo et al. 2026). Every system in Table 2-1 starts from labelled samples or threat reports; none asks whether an agent working from unlabelled sensor traffic can add detection to a strong human-tuned baseline.

Read critically, the reported results are also hard to compare. GenTI and RuleAutoPilot measure rules against the samples they were written from or against curated benchmarks, and FALCON validates syntax and semantics rather than detection on live traffic. None compares the generated rules with a tuned human baseline, and none controls for rules whose precision comes from the dataset's network layout rather than attack behaviour. Section V shows that this second effect can dominate an evaluation.

## Evaluation datasets and label quality

Network datasets differ widely in realism and labelling, so results on one dataset generalise poorly (Ring et al. 2019). CIC-IDS2017 labels every flow (Sharafaldin, Habibi Lashkari & Ghorbani 2018); UNSW-NB15 lists each attack event in nine categories (Moustafa & Slay 2015, 2016). Both have known label errors (Engelen, Rimmer & Joosen 2021; Lanvin et al. 2023), so a disagreement between a rule and the labels must be examined before it is counted as an error.

## Detection-engineering practice

In industry, rules come from a lifecycle rather than ad hoc writing. Palantir's Alerting and Detection Strategy framework requires each rule to document its goal, blind spots, expected false positives and validation (Palantir 2017). Atkinson (2020) argues that rules should target the attacker's capability, not the surface strings of one tool. Threat hunters find candidates by stack counting — counting every value of a field and examining the rarest — which needs no model or training period (Dragos 2023).

## Conclusions from the literature review

Signature detection is trusted but limited by human authoring; machine-learning detection is excluded here and is hard to operate. LLMs can draft rules, but only execution-validated rules can be trusted, and errors are confident. The gap this project addresses is the whole analyst loop — investigation to validated rule — from unlabelled traffic, measured against ground truth and against a strong human baseline, using the practices engineers already follow.

# III. Approach

The approach keeps the detection engine deterministic and automates the analyst around it. This section describes the architecture, the agent's tools, the technology stack, the safety design and the experiments.

## System architecture

Figure 3-1 shows the pipeline. Suricata 8.0.6 (OISF 2026) replays a packet capture with the ET Open ruleset (52,647 rules) and writes every event — alerts, flows, HTTP, DNS, TLS — to eve.json. For CIC-IDS2017 the sensor was first tuned on the benign Monday capture (529,918 flows): 80 documented suppressions bring Monday to zero alerts, as a SOC would tune a new sensor. The triage engine then hands the attack-day output to the LLM, and the rules it writes are loaded into the next replay. Rules are alert-only: a Suricata pass rule would exempt a whole flow from inspection, an avoidable bypass.

Each design choice is justified by a constraint or a finding. Suricata was chosen over Snort because it is multi-threaded, actively maintained and logs application-layer events as JSON, which the agent can query. Offline replay was chosen because it makes every experiment repeatable against labelled data and avoids sensor-placement and encrypted-traffic questions that are outside the scope. Starting from a mature community ruleset, rather than writing rules for every service from scratch, mirrors how a real SOC deploys a sensor and gives the AI a realistic baseline to improve on.

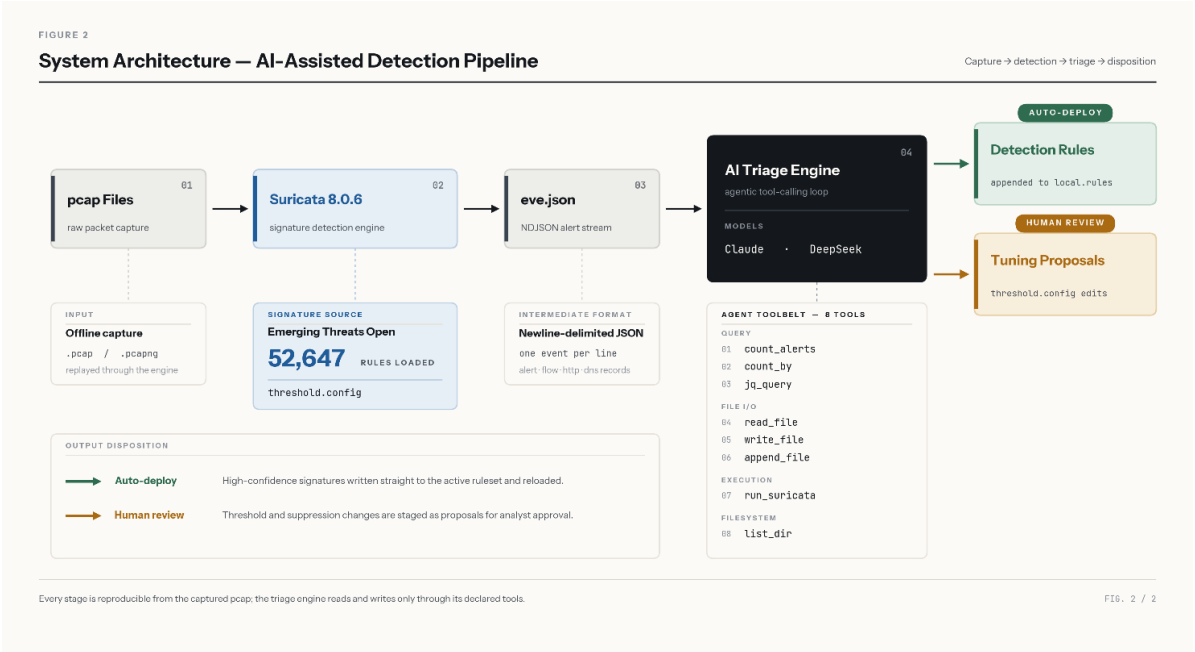



Figure 3-1: System architecture

## Agentic loop and provider adapter

A hosted LLM can only produce text. The engine turns it into an agent (Figure 3-2): each turn it sends the conversation and the tool schemas; the model replies with a tool call; the engine checks the call against an allowlist, runs it and returns the result. The loop follows the ReAct pattern (Yao et al. 2023) and ends when the model writes its final report. A six-step procedure in the system prompt — big picture, categorise, investigate, deep-dive, act, report — mirrors an analyst's workflow. A provider adapter lets DeepSeek (V3, V4 Pro) and Claude (Opus 5) run the identical loop, so the model can be swapped without changing the triage logic.

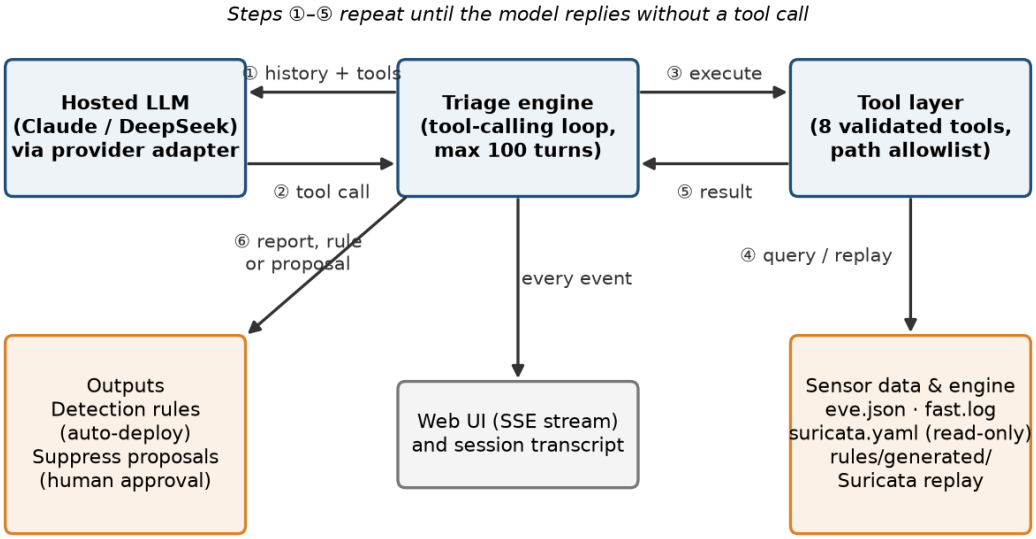



Figure 3-2: The agentic tool-calling loop

## The agent's tools

Tool design mattered as much as the choice of model. DeepSeek V3 initially failed because jq aggregation does not work on newline-delimited JSON; a purpose-built count_by tool, aggregating in Python, made the same model complete triage. The final toolkit has 13 tools in five groups (Figure 3-3): investigate (count_alerts, count_by, query_eve_json, read_file), hunt (stack_values), research (search_rules, lookup_vulnerability on a local NVD copy, suricata_keywords), validate (run_suricata, test_rule) and write (write_file, append_to_file). Each experiment profile enables only the tools its condition needs; the full tool table is Table D-1 in Appendix D.

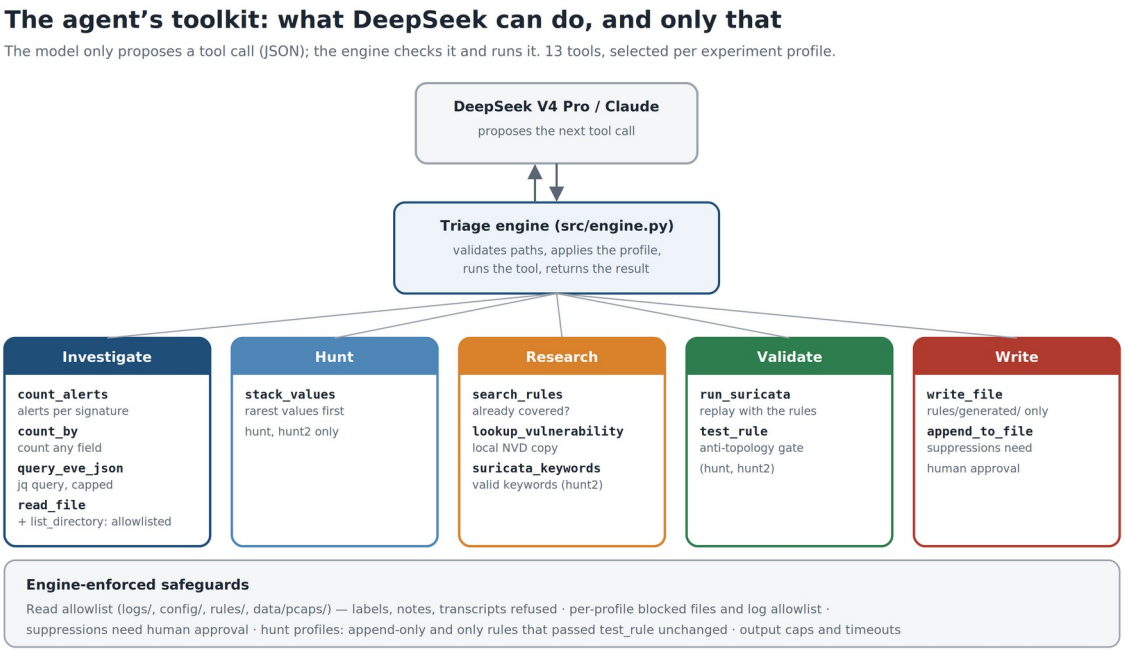



Figure 3-3: The agent's toolkit and engine-enforced safeguards

# Technology stack

Table 3-1 lists the stack. Everything is open source except the two hosted model APIs, which sit behind the adapter. DeepSeek V4 Pro was used for most runs because of its low cost.

Table 3-1: Technology stack

| **Component (version)** | **Role** |
|---|---|
| Suricata 8.0.6, suricata-update | Rule-based IDS: pcap replay, rule syntax checks, eve.json output |
| ET Open; Snort Community; Stamus; PT Open | Community rulesets for the baselines |
| Python 3.12 | Triage engine, tools, provider adapters, evaluation and figure scripts |
| jq 1.7; tshark / editcap 4.2 | Queries over eve.json; capture inspection and slicing |
| FastAPI + Uvicorn | Web UI streaming each reasoning step and tool call (Server-Sent Events) |
| DeepSeek V4 Pro / V3; Claude Opus 5 | Hosted LLMs via OpenAI-compatible and Anthropic SDKs |
| Local NVD database (SQLite) | Offline vulnerability lookup; no live web access |
| matplotlib, python-docx, pptxgenjs | Figures, this report and the presentation |
| Ubuntu 24.04 on WSL 2, Git | Development environment; one repository, four contributors |

# Safety design and evaluation integrity

The model has two exits (Figure 3-4). A confirmed threat becomes a detection rule, validated by replay and deployed automatically. Benign noise becomes a proposed suppression, which takes effect only after human

approval. The asymmetry is deliberate: a bad detection rule over-alerts, while a bad suppression silently blinds the sensor.

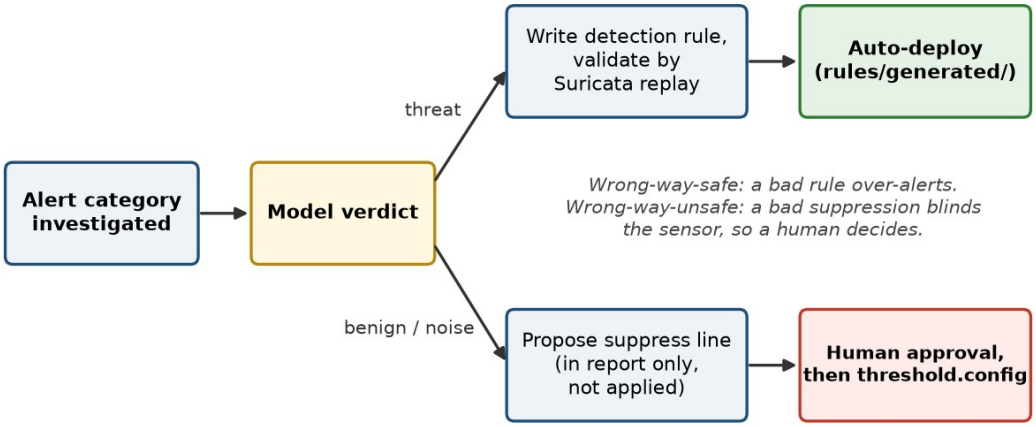



Figure 3-4: Safety asymmetry between detection and suppression

The model must never see ground truth. An audit of early transcripts found that two CIC runs had listed the label folder and one had read our notes, so the tools now open only logs, configurations, rules and captures; labels, notes, transcripts and results are refused. Each condition is a frozen, named experiment profile (Figure 3-5) fixing its configuration, prompt, tools, writable rule file and blocked files, so conditions cannot see each other's rules and every result traces to an exact set-up.

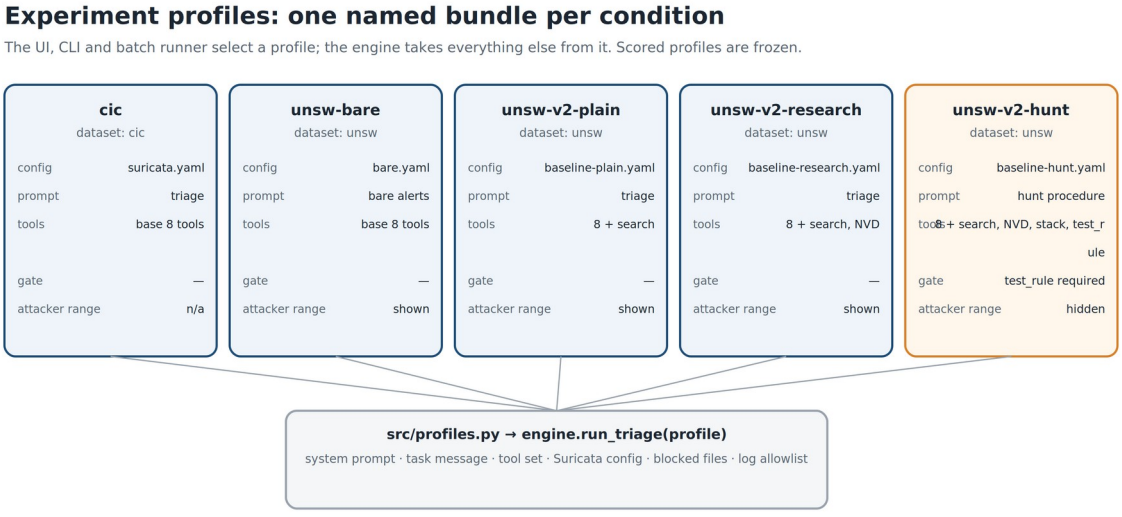



Figure 3-5: Experiment profiles

# Replicating detection-engineering practice: the hunt condition

Following the supervisor's advice to replicate what real engineers do, the hunt condition encodes the lifecycle from Section II as the agent's procedure (Figure 3-6). Instead of starting from alerts, the agent maps which protocols carry traffic but few alerts, stacks attacker-controlled fields rarest first, forms a hypothesis, writes a rule with an ADS-style comment, tests it and only then saves it. Its prompt omits the attacker address range, which a real engineer would not know.

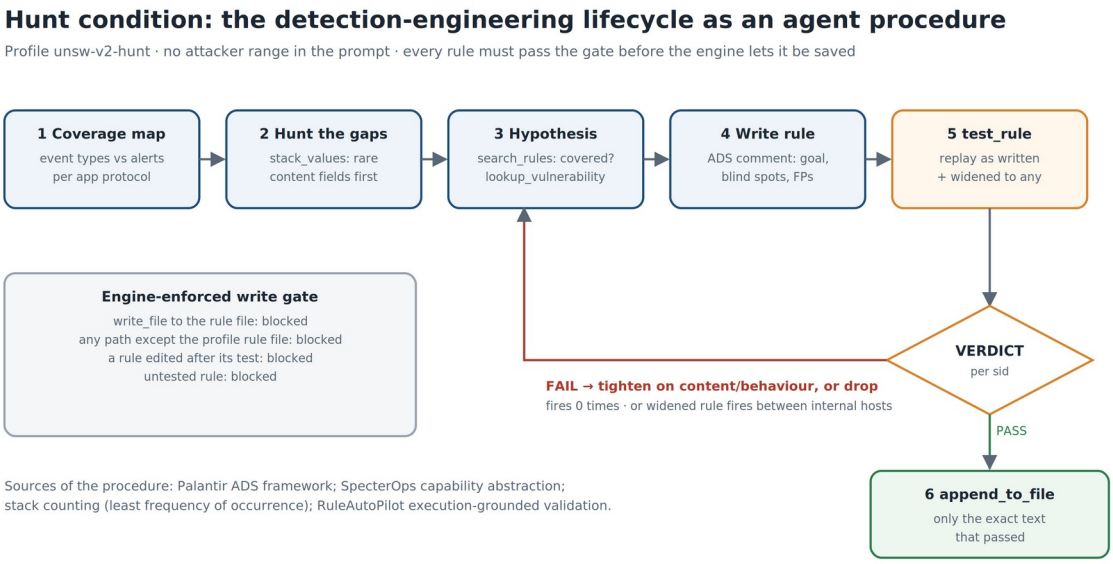



Figure 3-6: The hunt condition procedure

The test is test_rule (Figure 3-7). It replays a capture with the rule as written, then again with its addresses widened to any → any. If the widened rule fires between internal hosts, its precision came from the source address, not from attack behaviour, and it fails. Internal traffic is treated as known-good on the basis of the dataset's published testbed description (Moustafa & Slay 2015; Fraunhofer FKIE 2026). The engine refuses to save any rule that has not passed in exactly the form being saved.

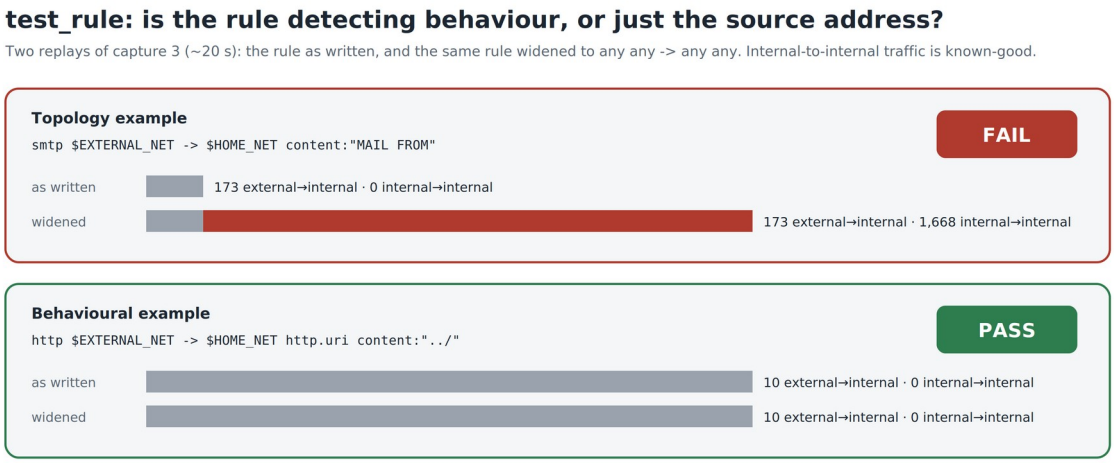



Figure 3-7: The test_rule anti-topology gate

# Experiments

Table 3-2 summarises the three experiments. Alerts were joined to labels by 5-tuple and time for CIC-IDS2017 (±120 s, because labels have minute resolution) and by endpoints, ports and time for UNSW-NB15 (±2 s); UNSW results are reported for TCP/UDP events.

Table 3-2: The three experiments

| Experiment | Question | Conditions | Measure |
|---|---|---|---|
| 1 — Context | What context is needed? | Full context vs alerts only | Categories, outcome evidence |
| 2 — Rule quality | Are AI rules good? | Stock vs stock + AI rules | Recall, precision, audit |
| 3 — Strong baseline | Can AI beat tuned rules? | Plain, research, hunt, hunt2 | Genuine gain, hold-out |

**Assumptions and limitations.** Offline replay stands in for a live sensor; one run per capture per condition means run-to-run variation is not separated from condition effects; and UNSW-NB15's layout, where every external host is an attacker, can make rules look better than they would be on a real network (Section V).

# IV. Execution

## Datasets

Table 4-1: Datasets used

| | **CIC-IDS2017** | **UNSW-NB15** |
|---|---|---|
| Source | Sharafaldin, Habibi Lashkari & Ghorbani (2018) | Moustafa & Slay (2015) |
| Captures | Mon (benign), Tue–Fri | Dev: 17 Feb (6 files); hold-out: 22 Jan (5 files) |
| Attack categories | 14 | 9 |
| Ground truth | Per flow | Per event (14,935 TCP/UDP dev) |
| Benign-only day | Yes (tuned sensor) | No (raw runs) |
| Role | Development | Generalisation; Experiment 3 |

Table 4-1 compares the datasets. UNSW-NB15 was added at the supervisor's request in Week 9 to test whether the approach generalises beyond the dataset it was developed on. The 22 January hold-out captures were not opened until every rule file had been frozen.

## CIC-IDS2017 triage

Each attack day was triaged twice: a tuned pass with the Monday suppressions and a raw pass without them. Figure 4-1 shows why both matter. Raw CIC captures produce 1.3–2.3 million alerts, mostly checksum engine events; tuning removes three to four orders of magnitude. Yet raw passes revealed things the tuned passes hid: a second reverse-shell victim and a Hulk-style HTTP flood with no stock rule.

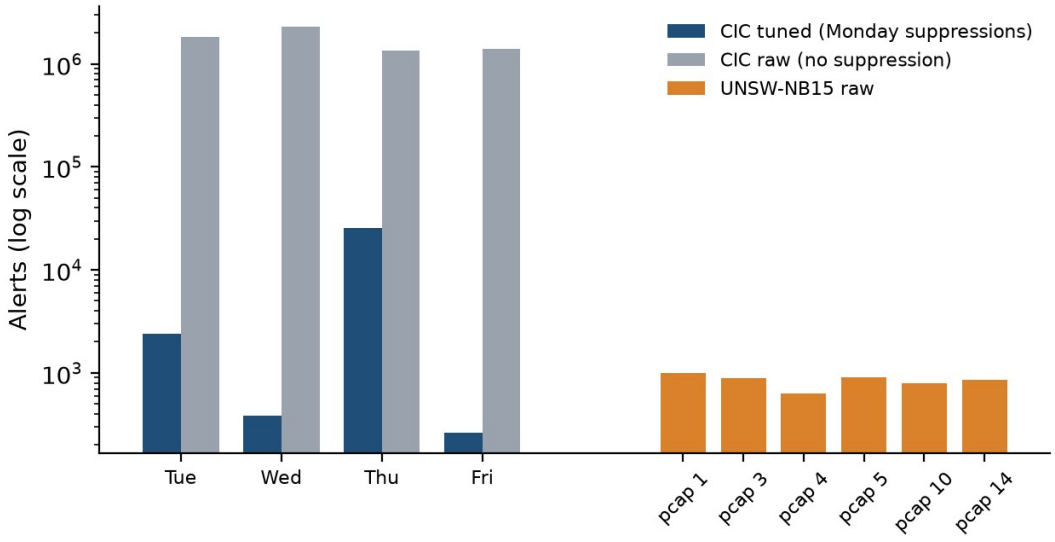



Figure 4-1: Alert volume per capture (log scale)

Across the four days the model identified 13 of the 14 labelled attack categories (only DoS GoldenEye was never named) and wrote 32 rule

definitions (Table D-3). Some show genuine analyst reasoning: DeepSeek V4 Pro diagnosed that Suricata's HTTP parser hides non-standard methods and rewrote a FAKEVERB rule on raw payload (0 → 463 alerts), and a later run found the ARES botnet the interim report had missed.

## UNSW-NB15 triage

The unchanged engine then triaged six UNSW-NB15 captures with DeepSeek V4 Pro. Each run took 43–75 turns and 9–111 minutes and produced 625–997 alerts. Across the six runs the model identified eight of the nine categories (Figure 4-2) — exactly in some cases, such as a single SQL Slammer worm event — and wrote 18 rules, all of which load and fire. In one run the model proposed suppressing two rules that were in fact firing on a Cisco exploit; because suppressions need approval the proposal was not applied, a direct test of the safety asymmetry.

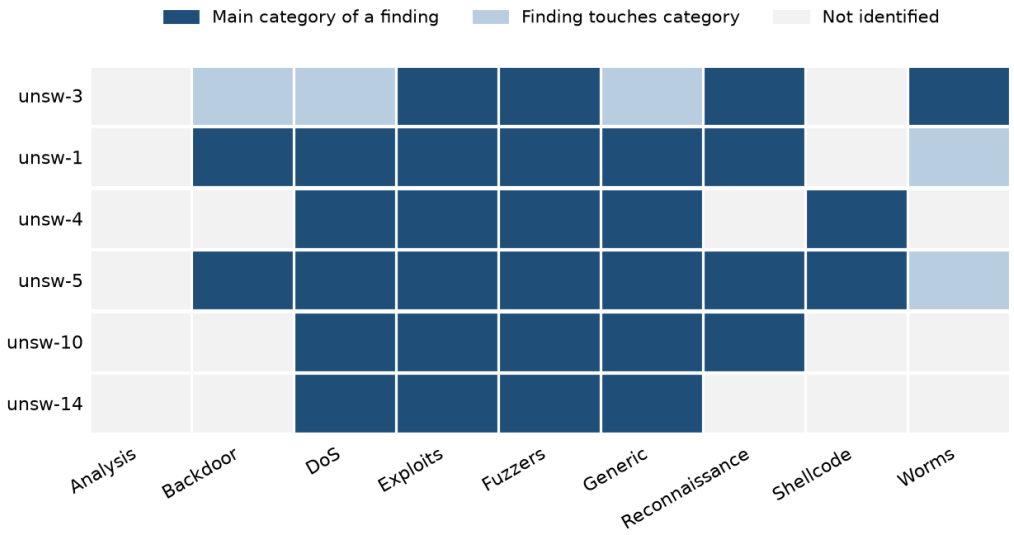



Figure 4-2: Categories identified per UNSW-NB15 run

## Strengthening the human baseline

A detection engineer's first step would be to tune the rules already available, so before any further AI rounds the UNSW-NB15 baseline was strengthened (Figure 4-3). Extra rulesets added almost nothing; re-enabling the rule families that ET Open and Snort ship disabled, chosen from the services under attack in the traffic and never from the labels, raised detection from 9.2% to 25.7%. Enabling every disabled rule reached 31.2% but raised alerts on hosts with no attacker involved from 809 to 37,405, 46 times the noise (Figure D-3). The selected set, baseline v2, was frozen.

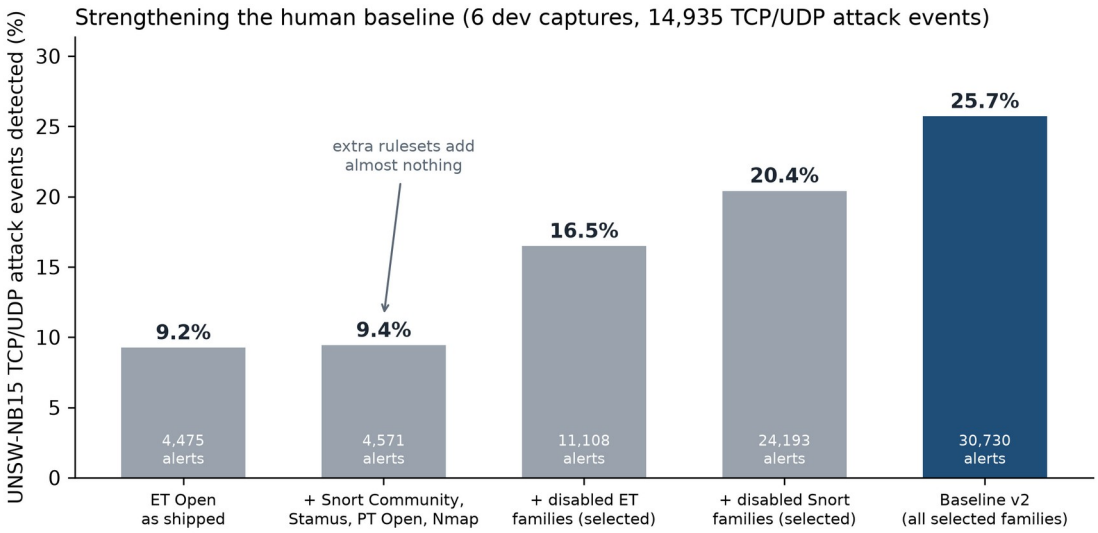



Figure 4-3: Strengthening the UNSW-NB15 baseline

# AI rounds on the strong baseline (Experiment 3)

Four conditions then wrote rules on captures 3, 1 and 10 against baseline v2: plain triage with a rule-search tool, research (adding vulnerability lookup), hunt, and hunt2 (hunt plus a keyword reference). All six development captures were replayed and scored after each round (Table 4-2).

Table 4-2: AI rounds on baseline v2 (14,935 events)

| Round | Rules (fired) | AI-rule alerts | Alerts matching an attack | Apparent gain (pts) | Genuine gain (pts) |
|---|---|---|---|---|---|
| Plain R2 | 16 (16) | 4,265 | 82.4% | +11.31 | +0.08 |
| Research R2 | 12 (11) | 2,193 | 88.3% | +2.68 | +0.33 |
| Hunt R1 | 25 (25) | 531 | 74.0% | +1.21 | +1.21 |
| Hunt2 R1 | 32 (32) | 677 | 85.4% | +0.96 | +0.91 |

The first hunt attempt on capture 1 failed instructively: rules passed the gate, but the agent kept re-testing them and hit the 100-turn limit without saving anything. Checking each layer — engine, server, browser, raw API — showed that DeepSeek V4 Pro was answering in thinking mode and our adapter discarded its reasoning, so the model began every turn without its own plan. With the reasoning carried, captures 1 and 10 finished in 90 and 93 turns and saved nine rules each.

The hunt reports read like an engineer's notes. On capture 3 the agent concentrated on HTTP, the protocol with the most external traffic and attacker-controlled fields, and its rarest user agents and URLs led to exploits the baseline missed: a D-Link router backdoor (CVE-2013-6026), three HP OpenView Network Node Manager CGI attacks and a PHP webshell drop. On capture 1 the gate taught it something about the engine: its traversal rule

on the decoded URI raised no alerts because Suricata normalises URLs, so it rewrote the rule on the raw URI and passed. It also declined to write rules it judged unreliable, such as randomised POP3 logins, and recorded why in its report.

# V. Analysis and Discussion of Results

## Rule quality against stock rules (Experiment 2)

On CIC-IDS2017 the AI rules add detection where stock rules had little or none (Figure 5-1): SSH-Patator rises from 1.0% to 28.4% of flows, and web brute force, the botnet and the Hulk flood go from zero to detected. Absolute recall is low for volumetric attacks because the AI rules fire once per campaign window while the labels count every flow.

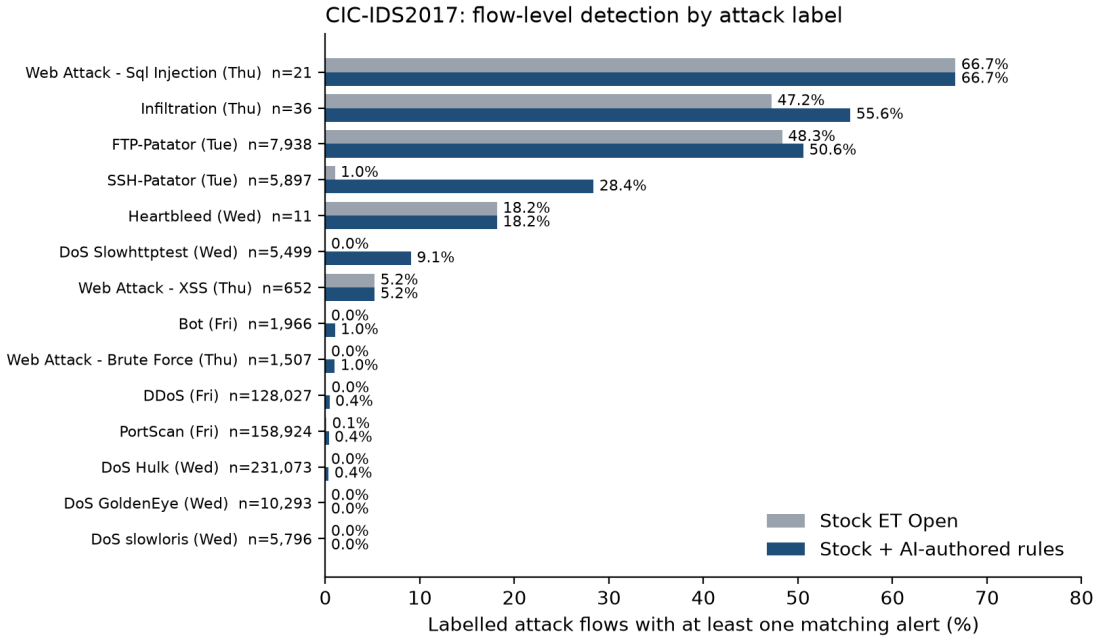



Figure 5-1: CIC-IDS2017 flow-level detection by attack

Precision is where the AI rules stand out (Figure 5-2): on the two noisiest captures 99.4% of their label-matched alerts fall on attack flows, against 6.1% for the stock rules. Category coverage rises from 8 to 12 of 14 on CIC-IDS2017 (Figure 5-3); on UNSW-NB15, a dataset the system was never tuned for, the AI rules raised event detection from 9.2% to 15.5% without a single alert on a benign host.

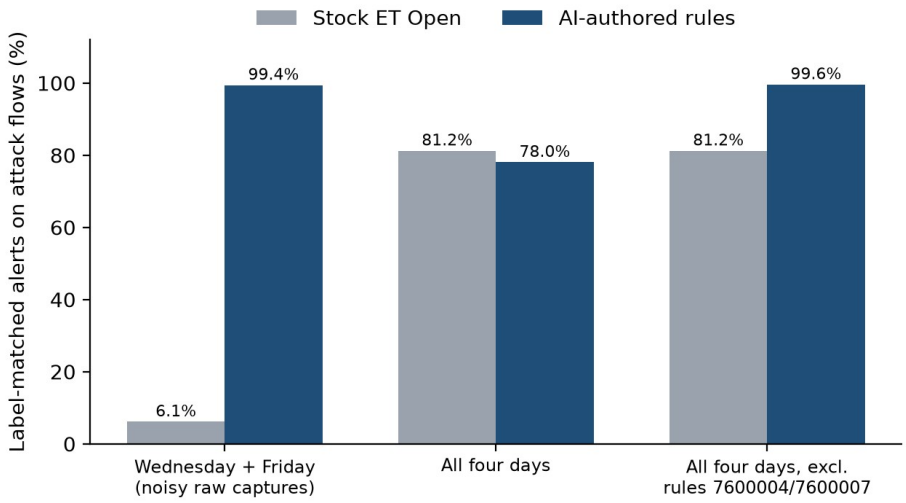



Figure 5-2: Precision of stock and AI-authored rules

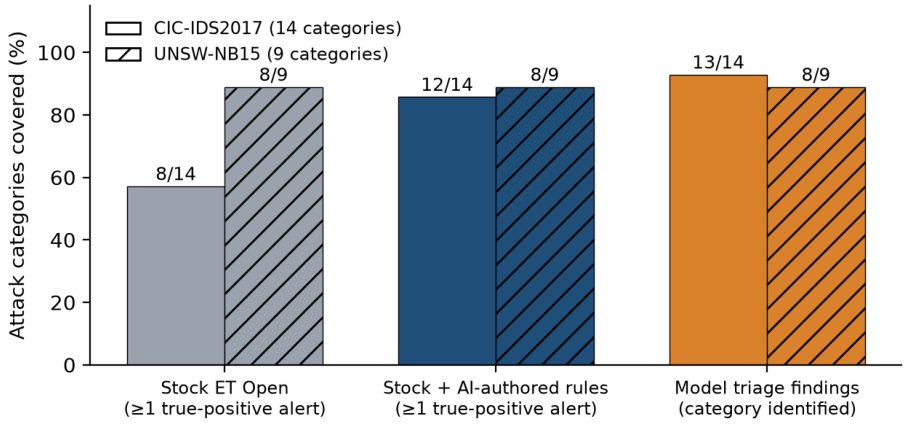



Figure 5-3: Attack categories covered

The rule audit (Table D-2, Figure D-1) tempers this. Of 32 CIC rule definitions, only 16 fired on replay; three use keywords Suricata rejects, one can never fire, and the model deleted two of its best rules on the false claim that they were stock ET rules. The cause was largely environmental: the CIC configuration loaded only three of ten generated rule files, so the model's validation reported success while testing nothing. On UNSW-NB15, where the configuration loaded its rule file, all 18 rules loaded and fired. An agent's self-validation is only as good as its environment.

## What context the model needs (Experiment 1)

Experiment 1 ran the same captures with alert events only. The result runs against expectation (Figure 5-4, Table 5-1): without context the model found attacks as broadly — seven categories against six. What context changed was depth. With every event type the model established what each attack achieved, in 52 evidence-backed outcome statements, against nine with alerts only. For an analyst, whether an attack succeeded decides the response, so context makes triage actionable.

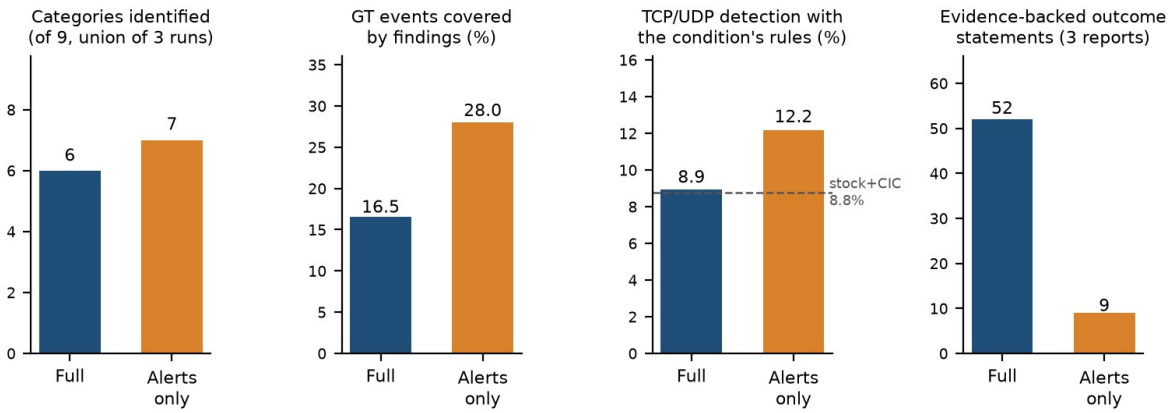



Figure 5-4: Experiment 1: full context versus alerts only

Table 5-1: Experiment 1 results

| **Measure (captures 3, 1, 10)** | **Full context** | **Alerts only** |
|---|---|---|
| Categories identified (union of runs, of 9) | 6 | 7 |
| Evidence-backed outcome statements | 52 | 9 |
| Rule alerts matching ground-truth events | 30% | 71% |
| Rule alerts on non-attacker hosts | 0 | 0 |

## Against a strong baseline: attribution and the topology trap

Experiment 3's first numbers looked strong: plain round 2 appeared to add 11 points. Two corrections removed most of it (Figure 5-5). First, Suricata replays are not repeatable, so comparing a fresh replay with a stored one counts timing noise as gain; gains are now credited only to events matched by an AI-rule alert and by no other alert in the same replay. Second, classifying every rule by its text (Figure 5-6) showed that most plain-round rules are *topology rules* — ordinary activity from outside the network, such as an SMTP MAIL FROM. In UNSW-NB15 every external host is an attacker, so such a rule scores perfectly here and would alert on every legitimate user of a real network. Counting only signature and behavioural rules, plain added +0.08 points and research +0.33.

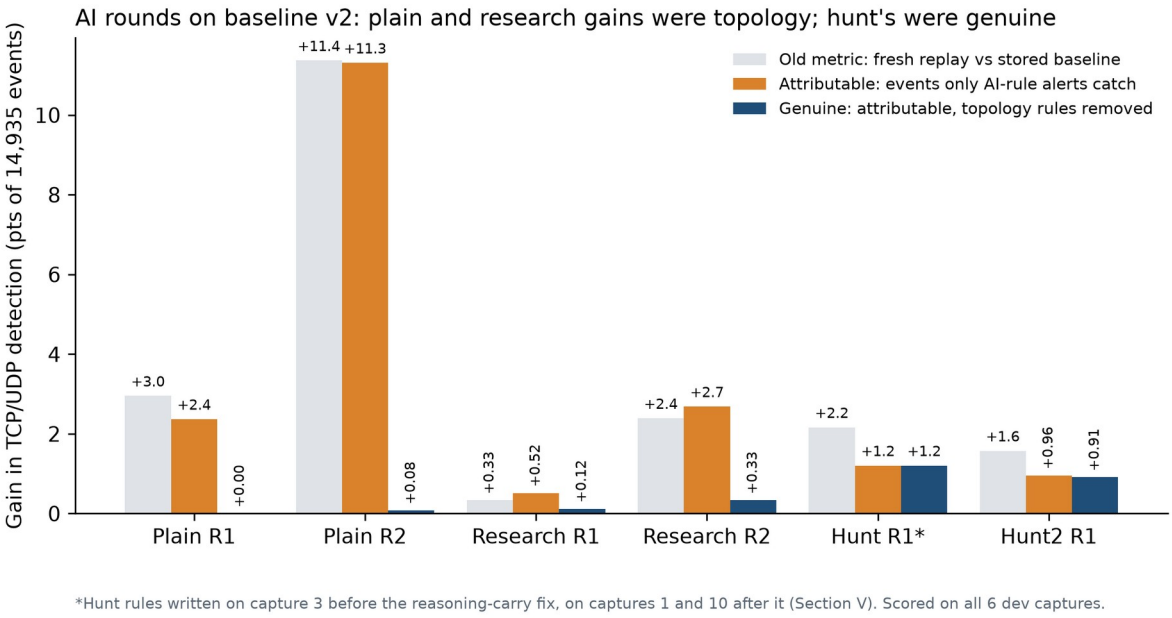



Figure 5-5: AI-round gains: apparent, attributable and genuine

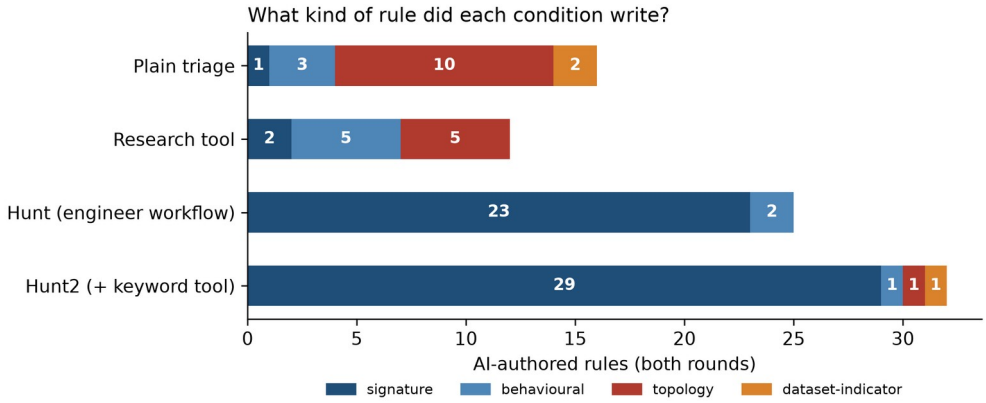



Figure 5-6: AI-authored rule classes by condition

The hunt condition wrote no topology rules. The gate rejected them as designed: widened, the MAIL FROM example matched 1,668 times between internal hosts, while a directory-traversal rule raised 10 alerts either way. Its 25 rules added +1.21 points, the first gain clearly above noise; hunt2, with the same procedure, added +0.91. Rules written before the reasoning fix keyed on one exploit attempt's filler and fired once (6 events); rules written after it targeted techniques — traversal, XSS, format strings, SMB fuzzing — and added 175 events, mostly on captures they were never written on (Figure D-2).

## Hold-out validation

Final claims rest on the 22 January captures, untouched until every rule file was frozen and scored once (Figure 5-7, Table 5-2). The frozen human baseline detected 29.5% of hold-out events, against 24.9% on the development captures, so the family selection generalises. The pattern of

the AI rounds held: hunt +1.53 points, hunt2 +1.06, research +0.28 and plain +0.11, with no AI-rule alert on the two benign-only captures in any condition. Together the four conditions add about +2.2 points.

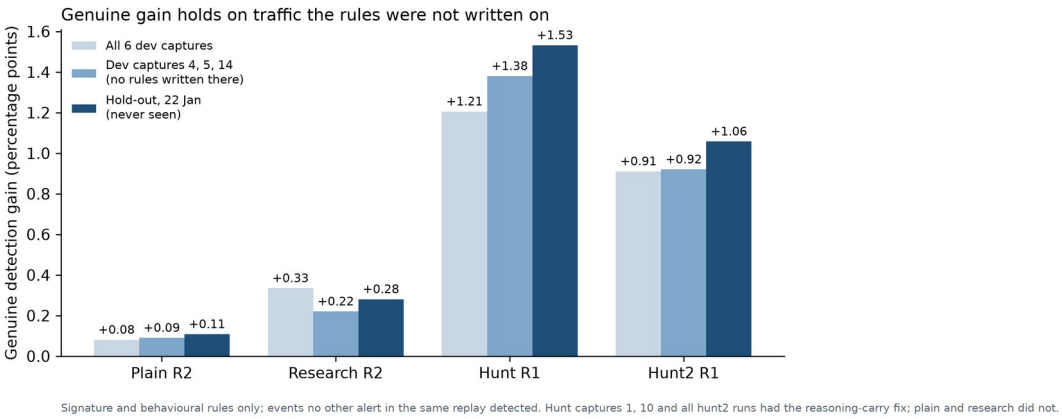



Figure 5-7: Genuine gain on development and hold-out captures

Table 5-2: Frozen AI rules on the hold-out (4,634 events)

| **Condition** | **Rules fired** | **AI-rule alerts** | **Matching an attack** | **Genuine gain** | **Alerts on benign captures** |
|---|---|---|---|---|---|
| Plain R2 | 14 of 16 | 966 | 98.3% | +5 (+0.11) | 0 |
| Research R2 | 10 of 12 | 835 | 84.0% | +13 (+0.28) | 0 |
| Hunt R1 | 17 of 25 | 155 | 91.6% | +71 (+1.53) | 0 |
| Hunt2 R1 | 22 of 32 | 177 | 100.0% | +49 (+1.06) | 0 |

# Discussion

Is 1.5 points small because signatures cannot see these attacks? No: 78.9% of development attack events carried data a content rule could match, and the baseline detects only 23.7% of those (Figure 5-8). The gap is unwritten signatures. Hunt2's keyword reference removed every rule-loading failure but did not raise detection, because two runs of the same procedure found largely different attacks (82 events shared of 180 and 136): what a run hunts matters more than syntax knowledge. The limits the agent itself reported are summarised in Figure D-4.

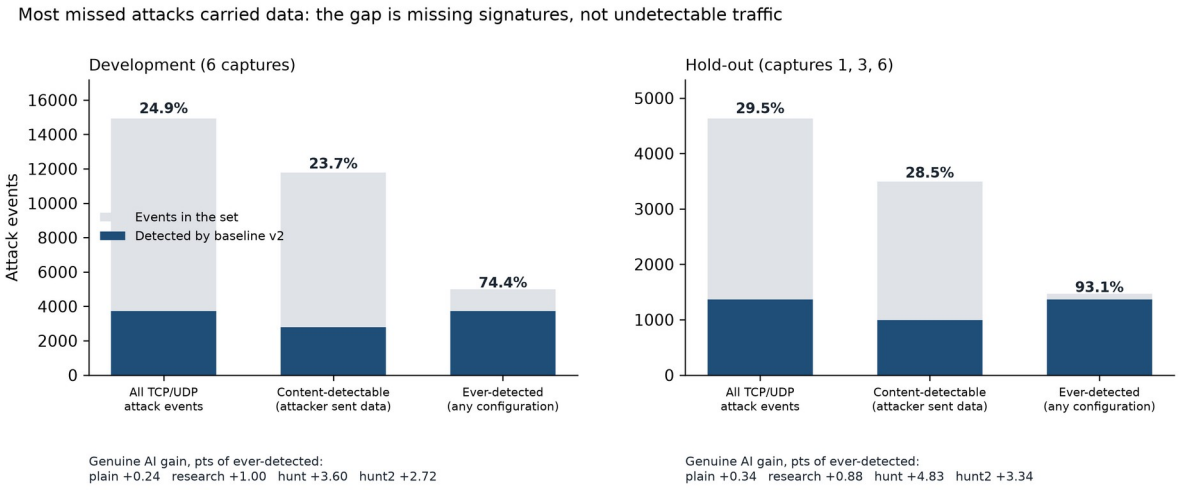



Figure 5-8: Detection against three denominators

The model's errors were confident: rules that do not load, routine FTP uploads read as data exfiltration, and opposite verdicts on the same evidence in different runs — the hallucination risk the literature predicts (Ji et al. 2023), and the reason di Filippo et al. (2026) found experts unwilling to use LLM rules unsupervised.

Set against the literature, the results support three points. First, execution-grounded validation is necessary but not sufficient: RuleAutoPilot-style checks would likely have passed most plain-round topology rules, because they fire on attack traffic and stay silent on benign traffic in this dataset. Only a test that removes the address constraint exposed them. Second, Sommer and Paxson's (2010) argument for automation that assists analysts holds for rule writing too: the agent's best contribution was a set of precise, documented, technique-level rules that a human can review in minutes, not an autonomous replacement. Third, tuning the human rules raised detection by 16 points while the best AI condition added 1.5, so a realistic evaluation must start from a tuned baseline, or it overstates what the AI adds.

**Threats to validity.** The hunt gain coincides with the reasoning fix, so the workflow and the fix cannot yet be separated. The hunt prompt also differs from triage in omitting the attacker range. Each condition ran once per capture. Rule classes were assigned from rule text. Early CIC runs could list the label folder, so the Tuesday triage is treated as label-informed. CIC labels contain known errors (Engelen, Rimmer & Joosen 2021).

# VI. Conclusions

Against the objectives: **O1** — the agentic engine completed the analyst workflow end to end on both datasets, with an enforced human gate on suppressions that stopped one harmful proposal. **O2** — scored against ground truth, its rules raised CIC-IDS2017 category coverage from 8 to 12 of 14 and UNSW-NB15 detection over stock ET Open from 9.2% to 15.5%, with 99.4% precision on the noisiest captures, but the audit found invalid, deleted and false-positive rules. **O3** — alert events alone were enough to find attacks; full sensor context is what let the model establish whether they succeeded. **O4** — against a strong human baseline, triage-driven rules added under half a point, while replicating detection-engineering practice added 1.2 points, and 1.5 on held-out traffic with no false alarms.

The answer to the research question is therefore split. For triage, yes: with sensor context an LLM can carry out the analyst's alert-handling work. For extending detection, partly: the gain is real and generalises, but it is modest against the roughly 70% of attacks every ruleset still misses, and the model's confident errors mean rule changes that reduce detection must stay under human approval.

The work has implications for practice. An LLM agent is already useful as a tireless first-line investigator that reads every alert and explains what happened, and as a drafter of precise rules that a human approves. It is not yet a safe autonomous rule manager. The most reliable gains came not from the model alone but from the engineering around it — tools, procedure, an enforced test gate and honest measurement.

# VII. Lessons Learned

- **Measure what the AI alone caught.** Replay variance and topology rules turned +0.08 into an apparent +11 points; we found it only by auditing our own scoring.
- **Tools and procedure shape the agent.** One aggregation tool rescued a failing model; one test gate eliminated topology rules. Better tools were worth more than a bigger model.
- **Debug the agent like any other system.** The hunt failure was a dropped API field, found by checking each layer.
- **Protect the evaluation early.** Label leakage in early runs forced re-scoring; access controls should come first.
- **Document as you go.** Writing alongside development, and freezing profiles and rule files before scoring, kept every number traceable when the scope grew with supervisor feedback.

# VIII. Future Work

- Re-run the plain and research conditions with reasoning carried, to separate the hunt workflow from the fix.
- Run each condition several times per capture and keep the union of genuine rules.
- Build the full context bundle (source history, asset context, rule history) and compare with a timed human analyst.
- Extend the test gate to rate rules and allow Lua rules for statistics such as DNS name entropy.
- Move from offline replay to a live sensor with time-windowed batching and live rule reloading.

# References


Anderson, JP 1980, *Computer security threat monitoring and surveillance*, Technical report, James P. Anderson Company, Fort Washington, Pennsylvania. http://csrc.nist.gov/publications/history/ande80.pdf

Aljanabi, M, Ismail, MA & Ali, AH 2021, 'Intrusion detection systems, issues, challenges, and needs', *International Journal of Computational Intelligence Systems*, vol. 14, no. 1, pp. 560–571. **https://doi.org/10.2991/ijcis.d.210105.001**

Anthropic 2025, 'Tool use and agentic workflows', *Claude Documentation*, viewed 15 September 2026, https://docs.anthropic.com/en/docs/build-with-claude/tool-use

Atkinson, J 2020, 'Capability abstraction', SpecterOps blog, 6 February, viewed 4 October 2026, https://specterops.io/blog/2020/02/06/capability-abstraction/

Axelsson, S 2000, 'The base-rate fallacy and the difficulty of intrusion detection', *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205. https://doi.org/10.1145/357830.357849

Buczak, AL & Guven, E 2016, 'A survey of data mining and machine learning methods for cyber security intrusion detection', *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

Denning, DE 1987, 'An intrusion-detection model', *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232. https://doi.org/10.1109/TSE.1987.232894

di Filippo, L, Bardhi, E, Agiollo, A, Palma, A, Bonomi, S & Kuipers, F 2026, 'Beyond the syntax: do security experts trust LLMs for NIDS rule engineering?', arXiv preprint arXiv:2607.05916.

Dragos 2023, 'Improving long tail analysis using Neighborhood Keeper', Dragos blog, 27 April, viewed 4 October 2026, https://www.dragos.com/blog/using-neighborhood-keeper-for-long-tail-analysis/

Engelen, G, Rimmer, V & Joosen, W 2021, 'Troubleshooting an intrusion detection dataset: the CICIDS2017 case study', in 2021 IEEE Security and Privacy Workshops (SPW), pp. 7–12. https://doi.org/10.1109/SPW53761.2021.00009

Ferrag, MA, Maglaras, L, Moschoyiannis, S & Janicke, H 2020, 'Deep learning for cyber security intrusion detection: Approaches, datasets, and

comparative study', *Journal of Information Security and Applications*, vol. 50, p. 102419. https://doi.org/10.1016/j.jisa.2019.102419

Fraunhofer FKIE 2026, 'UNSW-NB15', COMIDDS: Comprehensive Open Malware Intrusion Detection Dataset Store, viewed 4 October 2026, https://fkie-cad.github.io/COMIDDS/content/datasets/unsw_nb15/

Garcia-Teodoro, P, Diaz-Verdejo, J, Macia-Fernandez, G & Vazquez, E 2009, 'Anomaly-based network intrusion detection: Techniques, systems and challenges', *Computers & Security*, vol. 28, no. 1, pp. 18–28. https://doi.org/10.1016/j.cose.2008.08.003

Hadi, HJ, Yasmin, R & Shoker, A 2026, 'GenTI: Benchmarking LLMs for Autonomous IDPS Rule Generation for Unseen Attacks', arXiv preprint, arXiv:2606.05844.**https://doi.org/10.48550/arXiv.2606.05844**

Ji, Z, Lee, N, Frieske, R, Yu, T, Su, D, Xu, Y, Ishii, E, Bang, YJ, Madotto, A & Fung, P 2023, 'Survey of hallucination in natural language generation', ACM Computing Surveys, vol. 55, no. 12, art. 248. https://doi.org/10.1145/3571730

Khraisat, A, Gondal, I, Vamplew, P & Kamruzzaman, J 2019, 'Survey of intrusion detection systems: techniques, datasets and challenges', *Cybersecurity*, vol. 2, no. 1, p. 20.https://doi.org/10.1186/s42400-019-0038-7

Ko, C, Ruschitzka, M & Levitt, K 1997, 'Execution monitoring of security-critical programs in distributed systems: A specification-based approach', in *Proceedings 1997 IEEE Symposium on Security and Privacy*, pp. 175–187, IEEE. https://doi.org/10.1109/SECPRI.1997.601332

Lanvin, M, Gimenez, PF, Han, Y, Majorczyk, F, Mé, L & Totel, É 2023, 'Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes', in Risks and Security of Internet and Systems (CRiSIS 2022), Lecture Notes in Computer Science, vol. 13857, Springer, pp. 18–33. https://doi.org/10.1007/978-3-031-31108-6_2

Li, J, Chai, Y, Du, L, Duan, C, Yan, H & Gu, Z 2025, 'GRIDAI: generating and repairing intrusion detection rules via collaboration among multiple LLM-based agents', arXiv preprint arXiv:2510.13257.

Liao, HJ, Lin, CHR, Lin, YC & Tung, KY 2013, 'Intrusion detection system: A comprehensive review', *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24. https://doi.org/10.1016/j.jnca.2012.09.004

Mitra, S, Neupane, S, Duclos, M, Mittal, S, Piplai, A, Rahman, MR, Zieglar, E & Rahimi, S 2025, 'FALCON: autonomous cyber threat intelligence mining with LLMs for IDS rule generation', arXiv preprint arXiv:2508.18684.

Moustafa, N & Slay, J 2015, 'UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)', in 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, pp. 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

Moustafa, N & Slay, J 2016, 'The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set', Information Security Journal: A Global Perspective, vol. 25, no. 1–3, pp. 18–31.

OISF 2026, *Suricata 8.0.6 documentation*, Open Information Security Foundation, viewed 15 September 2026, https://docs.suricata.io/en/suricata-8.0.6/.

Palantir 2017, Alerting and detection strategy framework, GitHub repository, Palantir Technologies, viewed 4 October 2026, https://github.com/palantir/alerting-detection-strategy-framework

Paxson, V 1999, 'Bro: A system for detecting network intruders in real-time', *Computer Networks*, vol. 31, no. 23, pp. 2435–2463. https://doi.org/10.1016/S1389-1286(99)00112-7

Proofpoint 2026, *Emerging Threats Open ruleset*, Proofpoint Inc., viewed 15 September 2026, https://rules.emergingthreats.net/open/ .

Rehman, MU, Piplai, A & Kantarcioglu, M 2026, 'RuleAutoPilot: synthesizing deployable Suricata rules from network traffic', arXiv preprint arXiv:2609.16231.

Ring, M, Wunderlich, S, Scheuring, D, Landes, D & Hotho, A 2019, 'A survey of network-based intrusion detection data sets', Computers & Security, vol. 86, pp. 147–167. https://doi.org/10.1016/j.cose.2019.06.005

Roesch, M 1999, 'Snort: Lightweight intrusion detection for networks', in *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pp. 229–238. https://www.usenix.org/legacy/event/lisa99/full_papers/roesch/roesch.pdf

Scarfone, K & Mell, P 2007, *Guide to intrusion detection and prevention systems (IDPS)*, NIST Special Publication 800-94, National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-94

Schick, T, Dwivedi-Yu, J, Dessì, R, Raileanu, R, Lomeli, M, Hambro, E, Zettlemoyer, L, Cancedda, N & Scialom, T 2023, 'Toolformer: language models can teach themselves to use tools', in Advances in Neural Information Processing Systems, vol. 36.

Sekar, R, Gupta, A, Frullo, J, Shanbhag, T, Tiwari, A, Yang, H & Zhou, S 2002, 'Specification-based anomaly detection: a new approach for detecting network intrusions', in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pp. 265–274. https://doi.org/10.1145/586110.586146

Sharafaldin, I, Habibi Lashkari, A & Ghorbani, AA 2018, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', in *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pp. 108–116. https://doi.org/10.5220/0006639801080116

Sommer, R & Paxson, V 2010, 'Outside the closed world: On using machine learning for network intrusion detection', in *2010 IEEE Symposium on Security and Privacy*, pp. 305–316, IEEE. https://doi.org/10.1109/SP.2010.25

Vaswani, A, Shazeer, N, Parmar, N, Uszkoreit, J, Jones, L, Gomez, AN, Kaiser, Ł & Polosukhin, I 2017, 'Attention is all you need', in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008. https://doi.org/10.48550/arXiv.1706.03762

Yao, S, Zhao, J, Yu, D, Du, N, Shafran, I, Narasimhan, K & Cao, Y 2023, 'ReAct: synergizing reasoning and acting in language models', in Proceedings of the 11th International Conference on Learning Representations (ICLR 2023).

Zhang, X, Du, X, Chen, H, He, Y, Niu, W & Li, Q 2025, 'Automatically generating rules of malicious software packages via large language model', in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 734–747, IEEE. https://doi.org/10.1109/DSN64029.2025.00072

# Appendix A: Turnitin Similarity Report

[To be inserted: upload the final report, excluding the References section, to the Turnitin checker on Learnline (closes 25 October 2026) and insert the generated similarity report here.]

# Appendix B: AI Declaration Form

[To be completed by the group on the AI Declaration Form template from Learnline. Generative AI tools were used in this project in two ways that must be declared: (1) as the subject of the research — DeepSeek V3/V4 Pro and Claude Opus 5 are the models under test, called through their APIs by the triage engine; and (2) as development and writing assistance — state honestly which tools were used for code, figures and drafting of this report, and for which sections. Under the unit rules, any AI-generated text kept verbatim must be in italics and referenced.]

# Appendix C: Data and Source Code

All code, configuration, AI-authored rules, results and session transcripts are in the project's Git repository [insert repository URL], laid out as in Table C-1. Large packet captures and logs are not committed; they are public and can be re-downloaded.

Table C-1: Repository layout

| **Path** | **Contents** |
|---|---|
| src/ | Triage engine (engine.py), tools (tools.py), prompts and experiment profiles, provider adapters, web server and UI |
| src/eval/ | Ground-truth scoring for both datasets, rule audit, rounds and hold-out scoring, figure scripts |
| config/ | Suricata configurations: CIC (frozen) and UNSW profiles including baseline v2 |
| rules/generated/, rules/snapshots/ | Every AI-authored rule file, and the frozen copies scored on the hold-out |
| docs/results/ | All numbers in this report as JSON/CSV |
| triage-*.txt | Session transcripts — the model's reasoning and every tool call |
| Datasets | CIC-IDS2017: unb.ca/cic/datasets/ids-2017.html; UNSW-NB15: research.unsw.edu.au/projects/unsw-nb15-dataset |

To reproduce: install Suricata 8.0.6 and the Python requirements in a virtual environment; replay a capture with scripts/run_suricata.sh; start the UI with `python -m src.server` (a DeepSeek or Anthropic API key in the environment); score with the scripts in src/eval/.

# Appendix D: Supplementary Material

Detail removed from the body to meet the length limit: the full tool list (Table D-1), the rule audit (Table D-2, Figure D-1), the CIC-IDS2017 execution summary (Table D-3), the transfer of each hunt rule across captures (Figure D-2), the all-rules-enabled control (Figure D-3) and the reasons the agent left activity without a rule (Figure D-4).

Table D-1: The agent's tools

| **Tool** | **What it does** | **Safeguard** | **Profiles** |
|---|---|---|---|
| count_alerts | Alerts grouped by signature | Allowlisted paths | All |
| count_by | Counts values of any field (Python, works on NDJSON) | Allowlist; 120 s timeout | All |
| query_eve_json | jq filter over eve.json | 150-line cap; 60 s timeout | All |
| read_file / list_directory | Read configs, rules, logs | Labels, notes, transcripts, results refused | All |
| write_file / append_to_file | Write a rule file or propose a suppression | rules/generated only; suppressions need approval; hunt: append only, tested rules only | All |
| run_suricata | Replay a capture with the profile's configuration | Profile config only; 10 min timeout | All |
| search_rules | Is the behaviour already covered by a loaded rule? | Read-only | Plain, research, hunt, hunt2 |
| lookup_vulnerability | Local NVD CVE lookup | Offline; refuses dataset terms | Research, hunt, hunt2 |
| stack_values | Field values, rarest first | Allowlist; capped | Hunt, hunt2 |
| test_rule | Syntax check; replay as written and widened; PASS/FAIL | Counts by host role only | Hunt, hunt2 |
| suricata_keywords | Keywords this Suricata build supports | Read-only | Hunt2 |

Table D-2: Audit of AI-authored rule definitions (Experiments 1–2)

| **Rule status** | **CIC-IDS2017** | **UNSW-NB15** |
|---|---|---|
| Fired on replay | 16 | 18 |
| Replayed, never fired | 4 | 0 |
| Never replayed (file not loaded) | 6 | 0 |
| Loads, can never fire | 1 | 0 |
| Fails to load | 3 | 0 |
| Deleted by the model | 2 | 0 |
| Total rule definitions | 32 | 18 |

Table D-3: CIC-IDS2017 execution summary

| **Day** | **Model(s)** | **Tuned / raw alerts** | **Attacks identified** | **Rules** |
|---|---|---|---|---|
| Tue | Claude Opus 5, DeepSeek V3 | 2,404 / 1,842,712 | FTP and SSH brute force | 7 |
| Wed | DeepSeek V3, V4 Pro | 381 / 2,310,105 | DoS Hulk, Slowloris, | 5 (+2 deleted) |

| Day | Model(s) | Tuned / raw alerts | Attacks identified | Rules |
|---|---|---|---|---|
| | | | Slowhttptest, Heartbleed | |
| Thu | DeepSeek V4 Pro | 25,406 / 1,346,596 | Web brute force, XSS, SQLi, infiltration | 7 |
| Fri | DeepSeek V4 Pro | 263 / 1,412,449 | DDoS, port scan, ARES botnet | 11 (incl. copies) |

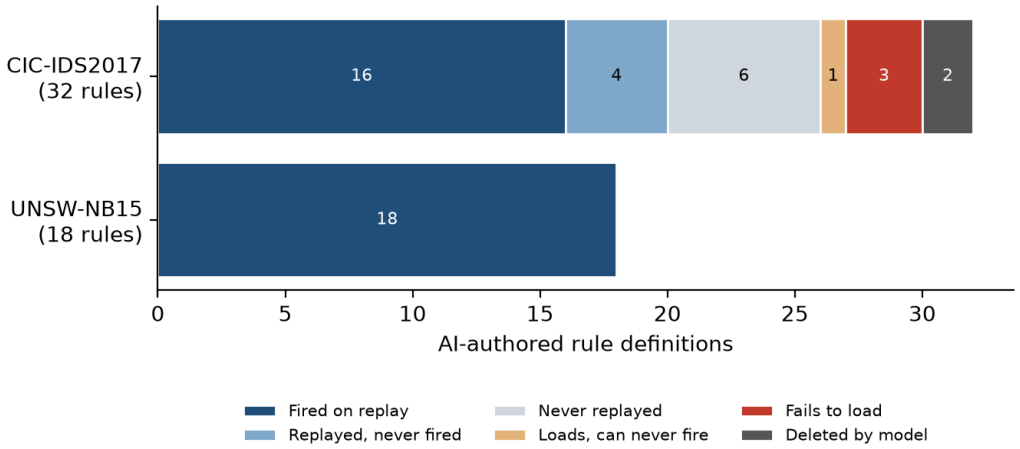



Figure D-1: Status of every AI-authored rule definition

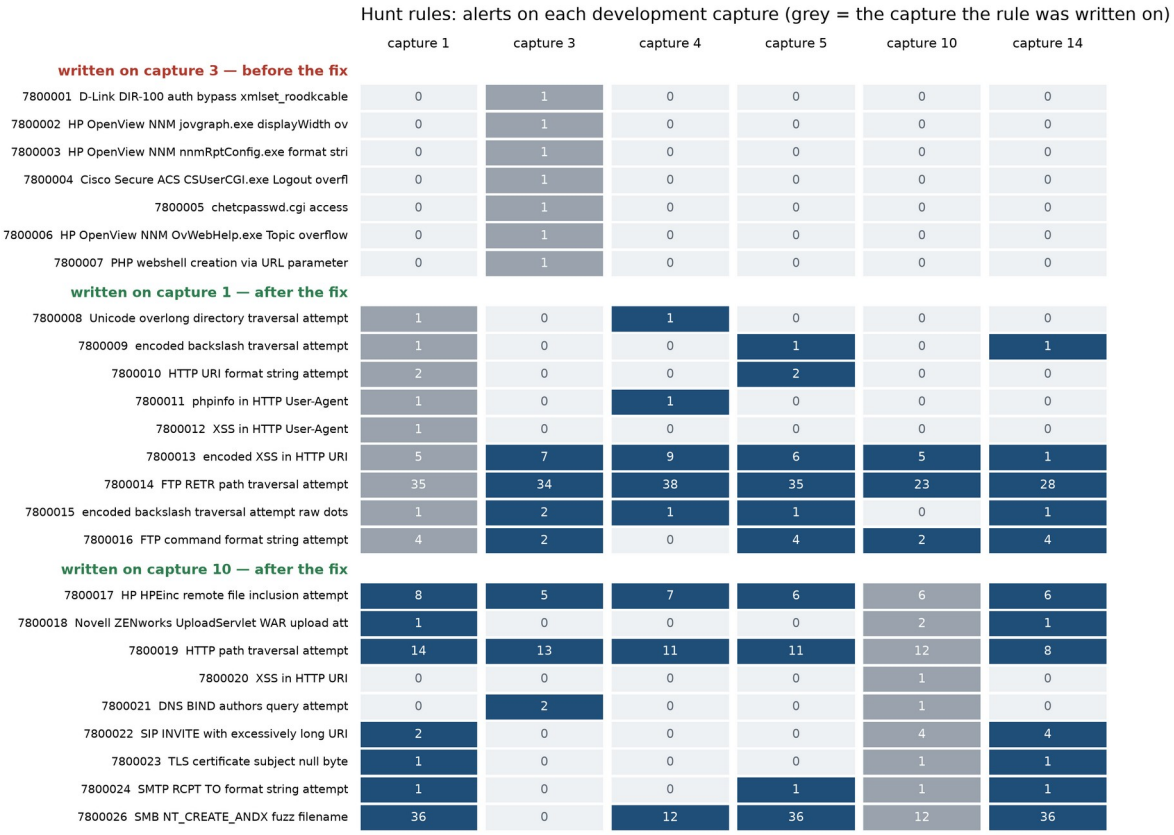



Figure D-2: Alerts of each hunt rule on each development capture, grouped by the capture it was written on

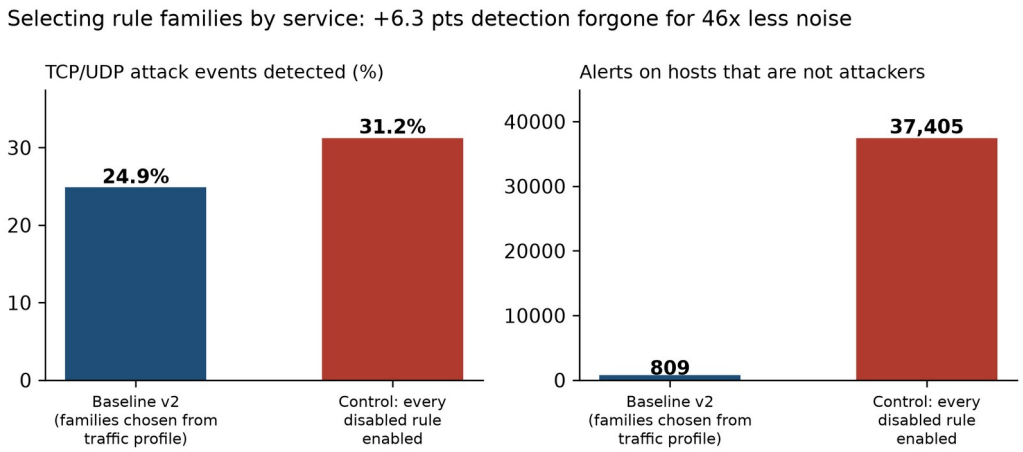



Figure D-3: Selected rule families versus enabling every disabled rule — detection and alerts on non-attacker hosts

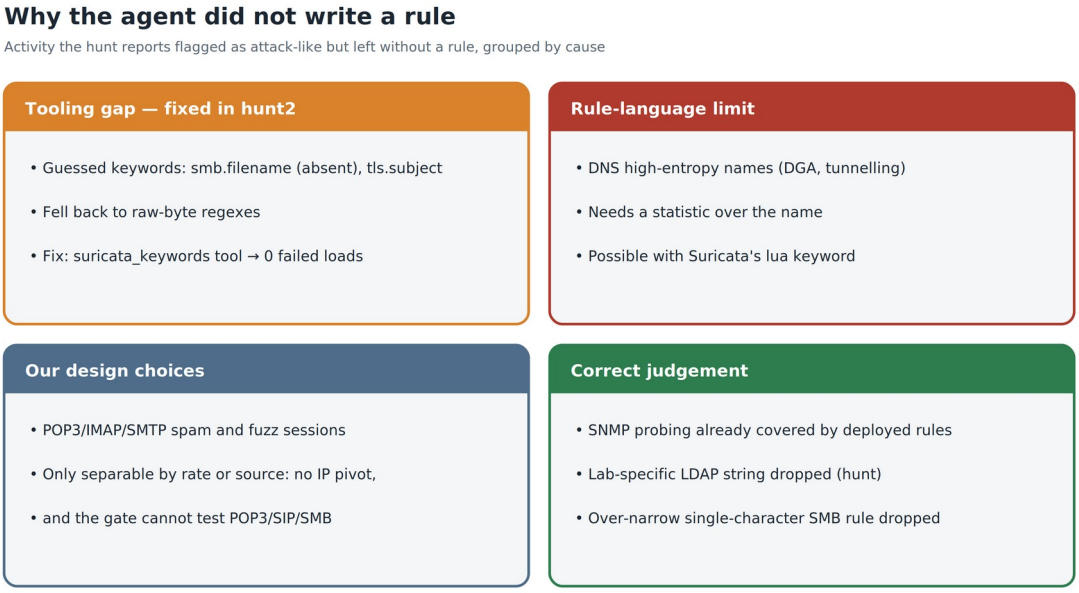



Figure D-4: Attack-like activity the hunt reports left without a rule, grouped by cause

The complete 81-page development report, with every table and transcript reference, is kept in the repository as docs/Final_Report_Group13_DRAFT.docx.